

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO BÀI TẬP LỚN
LẬP TRÌNH MẠNG

Đề tài: Ironfront Reborn – Trò chơi bắn súng nhiều người chơi
với tầng mạng TCP/UDP tự xây dựng

Lớp: D23CTPM01-B

Nhóm bài tập lớn: 10

Sinh viên thực hiện: Nguyễn Thành Trung

Mã sinh viên: B23DCCN861

Báo cáo kiến trúc và giao diện hệ thống

Phần cá nhân: *Phân hệ client Unity*

Hà Nội – 2026

Mục lục

I. NỘI DUNG NHÓM	3
1. Mô tả kiến trúc chung của hệ thống	3
1.1. Bài toán và phạm vi	3
1.2. Kiến trúc ba tiến trình	3
1.3. Phân chia TCP và UDP theo đặc tính dữ liệu	5
1.4. Kiến trúc phân lớp mã nguồn	5
1.5. Kiến trúc triển khai	6
1.6. Phân công theo phân hệ	7
2. Mô tả thiết kế chung của hệ thống	7
2.1. Mô hình thẩm quyền	7
2.2. Luồng nghiệp vụ đầu – cuối	8
2.3. Các kỹ thuật netcode cốt lõi	9
2.4. Thiết kế giao thức	9
2.5. Tham số thời gian	10
2.6. Phân loại đối tượng đồng bộ	11
2.7. Bảo mật ở mức phù hợp phạm vi	11
2.8. Các quyết định kiến trúc đã chốt	11
2.9. Quy trình phát triển và bảo đảm chất lượng	11
II. NỘI DUNG CÁ NHÂN – Phân hệ client Unity.	13
3. Biểu đồ và giao diện nội dung cá nhân thực hiện	13
3.1. Điểm nối netcode trong mã gameplay	13
3.2. Thứ tự thực thi một khung hình client	14
3.3. Máy trạng thái luồng trò chơi	14
3.4. Giao diện sảnh chờ	15
3.5. Giao diện trong trận và overlay chẩn đoán	16
3.6. Luồng dự đoán và hoà giải phía client	16
4. Luồng xử lý và giải pháp đã ứng dụng	16
4.1. Luồng đăng nhập từ giao diện	17
4.2. Luồng vào phòng và kết nối game server	18
4.3. Luồng dự đoán và hoà giải chuyển động người chơi cục bộ	19
4.4. Luồng hiển thị người chơi và bot ở xa	20
4.5. Luồng sự kiện chiến đấu, mục tiêu và HUD	21
4.6. Luồng mất kết nối và kết thúc trận	21

I. NỘI DUNG NHÓM

1. Mô tả kiến trúc chung của hệ thống

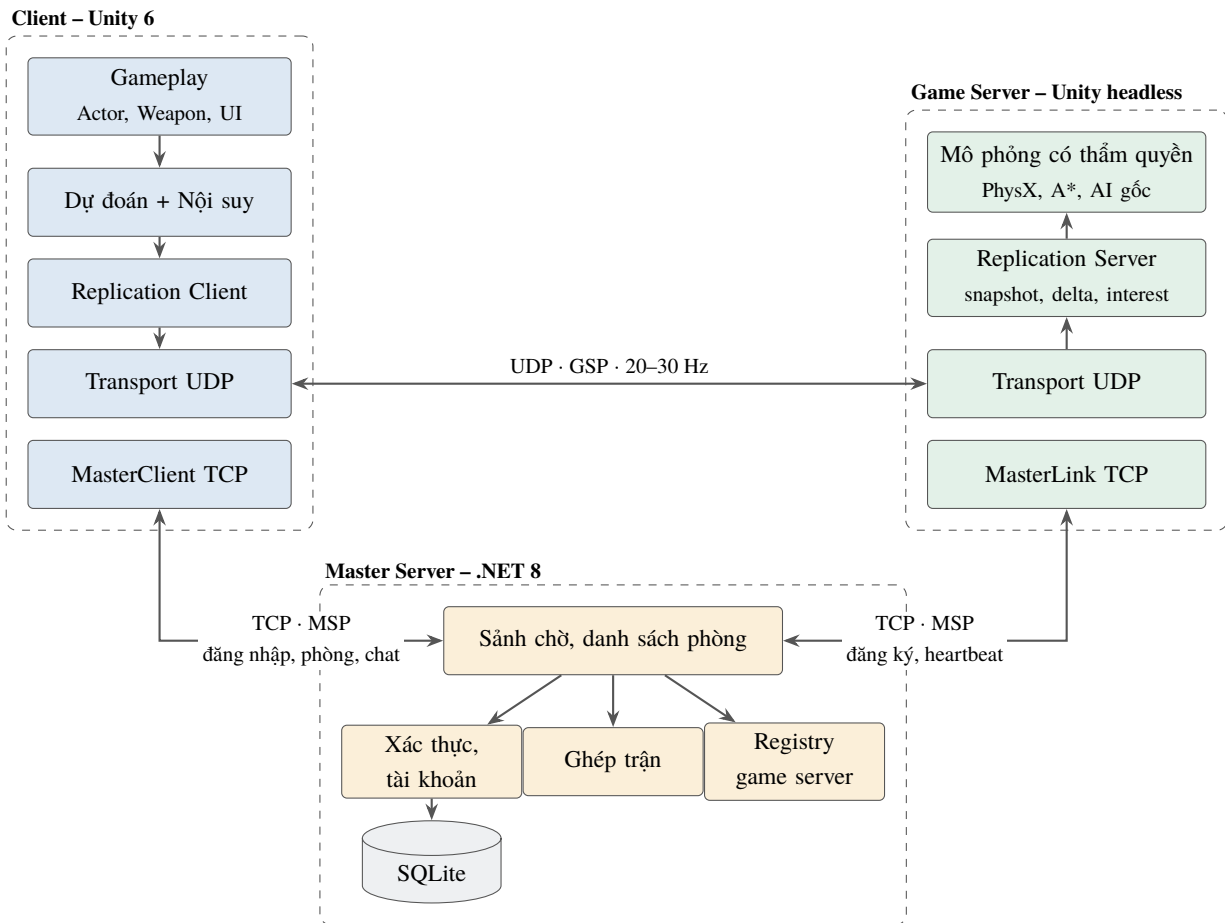
1.1. Bài toán và phạm vi

Ironfront Reborn chuyển mã nguồn trò chơi bắn súng góc nhìn thứ nhất *Ravenfield Beta 5* (Unity 6000.3, vốn chỉ chơi đơn) thành một trò chơi **nhiều người chơi, máy chủ nắm quyền quyết định** (server-authoritative). Yêu cầu cốt lõi của môn Lập trình mạng: **toàn bộ tầng mạng được viết tay trên socket TCP và UDP** thô qua `System.Net.Sockets` — không dùng Mirror, Photon, Netcode for GameObjects, KCP, gRPC hay WebSocket.

Tiêu chí	Giá trị thiết kế
Quy mô một trận	16 người chơi thật + 32 bot AI (tối đa 64 actor, 16 phương tiện)
Tần số mô phỏng / snapshot / gửi input	30 Hz / 20 Hz / 30 Hz
Băng thông xuống mỗi client	≤ 8 KB/s (do trên server đã triển khai: 1,67 KB/s)
Triển khai	LAN trước, sau đó VPS công cộng (Docker Compose, TLS cho kênh TCP)
Công nghệ	.NET 8 (master server, công cụ), netstandard2.1 (thư viện dùng chung), Unity 6 (client và game server headless)

1.2. Kiến trúc ba tiến trình

Hệ thống gồm **ba loại tiến trình** độc lập, giao tiếp qua **hai giao thức tự định nghĩa**: **MSP** (Master Server Protocol, trên TCP) cho dịch vụ sảnh chờ và **GSP** (Game Server Protocol, trên UDP) cho lưu lượng trong trận. Hai giao thức gặp nhau ở đúng một điểm: **joinTicket** được master ký HMAC và game server tự kiểm tra.



Hình 1.1. Kiến trúc tổng thể ba tiến trình và hai giao thức MSP/GSP

Tiến trình	Nền tảng	Trách nhiệm chính	Giao tiếp
Client	Unity 6, bản build thường	Hiển thị, nhận input, dự đoán chuyển động người chơi cục bộ, nội suy actor ở xa, giao diện sảnh và HUD	TCP tới Master; UDP tới Game Server
Game Server	Unity 6 headless (UNITY_SERVER)	Chạy mô phỏng có thẩm quyền (vật lý, AI, bắn, điểm chiếm), tạo snapshot riêng cho từng client, bù trễ	Nhận UDP từ client; TCP tới Master
Master Server	.NET 8 console, SQLite	Tài khoản, phiên, phòng, ghép trận, chat sảnh, registry game server, cấp joinTicket	Nhận TCP từ client và game server

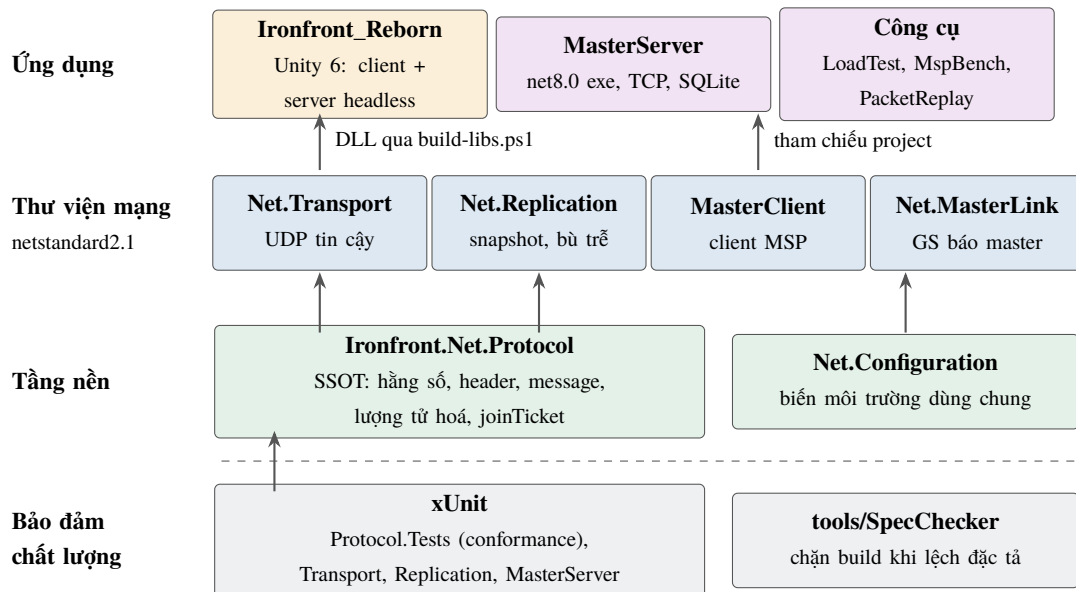
1.3. Phân chia TCP và UDP theo đặc tính dữ liệu

Đặc tính	TCP – Master Server	UDP – Game Server
Dùng cho	Đăng nhập, danh sách phòng, tạo/vào phòng, ghép trận, chat sảnh, heartbeat server	Mọi lưu lượng trong trận: input, snapshot, sự kiện gameplay
Tần suất	Vài gói/phút mỗi client	20–60 gói/giây mỗi chiều
Mất gói	Không chấp nhận (mất LOGIN_RES là treo màn hình)	Chấp nhận với trạng thái: snapshot sau thay thế snapshot trước
Nếu đổi chỗ	Phải tự viết lại độ tin cậy mà TCP đã làm tốt	Head-of-line blocking: một gói mất chặn mọi gói sau; Nagle + delayed ACK thêm 40–200 ms
Đóng khung	Tiền tố độ dài 4 byte + JSON	Datagram tự nhiên, header nhị phân 16 byte

Không dùng WebSocket: WebSocket chạy trên TCP nên thừa hưởng nguyên vẹn head-of-line blocking, cộng thêm chi phí bắt tay HTTP — một ràng buộc chỉ cần cho trình duyệt, không cần cho client desktop.

1.4. Kiến trúc phân lớp mã nguồn

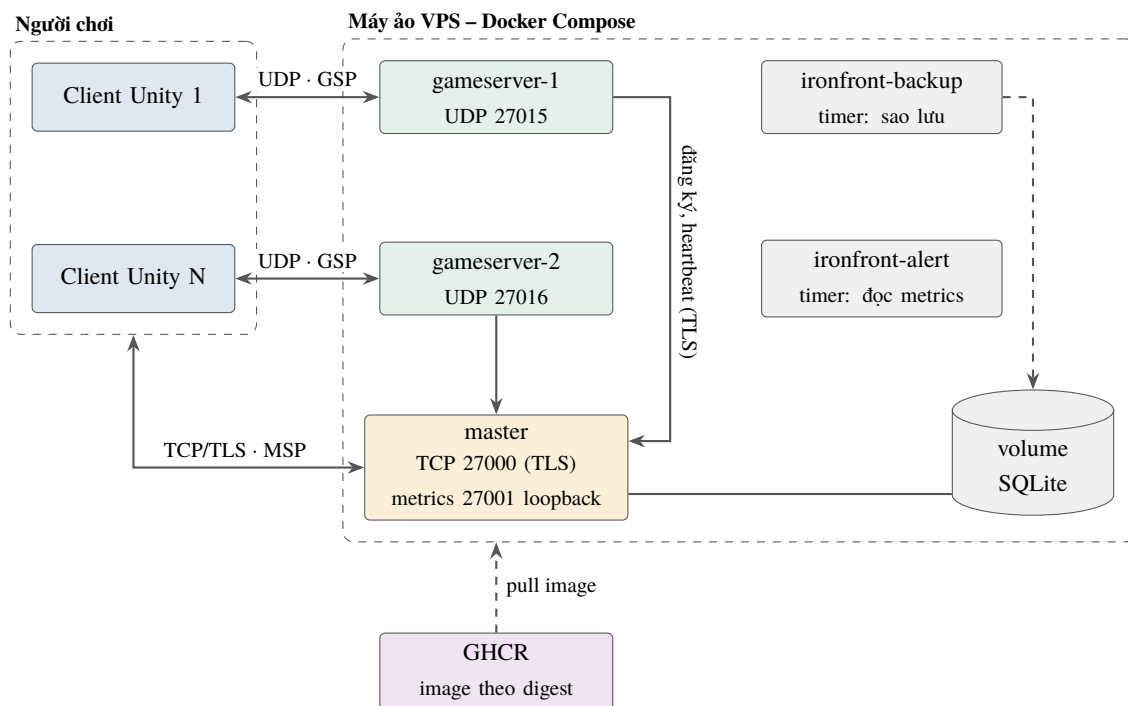
Mã nguồn chia thành các thư viện .NET thuần (netstandard2.1) **không bao giờ tham chiếu** UnityEngine; dự án Unity dùng chúng dưới dạng DLL do tools/build-libs.ps1 sinh vào Assets/Plugins. Nhờ vậy logic mạng được kiểm thử bằng xUnit trong vài giây, không cần mở Unity Editor.



Hình 1.2. Phân lớp thư viện và quan hệ phụ thuộc giữa các project trong solution

Project	Loại	Dòng mã	Vai trò
Ironfront.Net.Protocol	netstandard2.1	5 300	Nguồn sự thật duy nhất của định dạng dây
Ironfront.Net.Transport	netstandard2.1	4 100	Độ tin cậy trên UDP, kênh, phân mảnh, nghẽn, giả lập mạng
Ironfront.Net.Replication	netstandard2.1	18 500	Snapshot, delta, interest, tick server, bù trễ, trận đấu
Ironfront.MasterServer	net8.0 exe	4 300	Framing, dispatch, xác thực, sánh, ghép trận, TLS, metrics
MasterClient, MasterLink	netstandard2.1	1 250	API cho client Unity và liên kết game server – master
Assets/Scripts/Net	Unity C#	14 600	Lớp keo Unity: bootstrap, dự đoán, hiển thị, giao diện

1.5. Kiến trúc triển khai



Hình 1.3. Sơ đồ triển khai trên một máy ảo: một master và hai game server headless

Image được kéo từ GHCR theo digest cố định; bí mật (IRONFRONT_SHARED_SECRET, chứng chỉ TLS) lấy từ tệp .env không commit. Sao lưu và cảnh báo chạy bằng systemd timer *ngoài* container, vì bộ phát hiện sự cố phải nằm ngoài thứ nó giám sát. Hạ tầng còn có cấu hình Terraform (tạo VM, cloud-init) và fly.io làm phương án thay thế.

1.6. Phân công theo phân hệ

Sinh viên	Mã SV	Phân hệ	Phạm vi mã nguồn chính
Nguyễn Thành Trung	B23DCCN861	Client Unity	Assets/Scripts/Net/Client, Net/Input
Nguyễn Minh Toàn	B23DCCN832	Tầng giao vận UDP	Ironfront.Net.Transport
Nguyễn Duy Nghĩa	B23DCCN600	Replication, mô phỏng server	Ironfront.Net.Replication, Net/Server
Nguyễn Tự Kiên	B23DCCN465	Master Server TCP, hạ tầng	Ironfront.MasterServer, MasterClient, infra

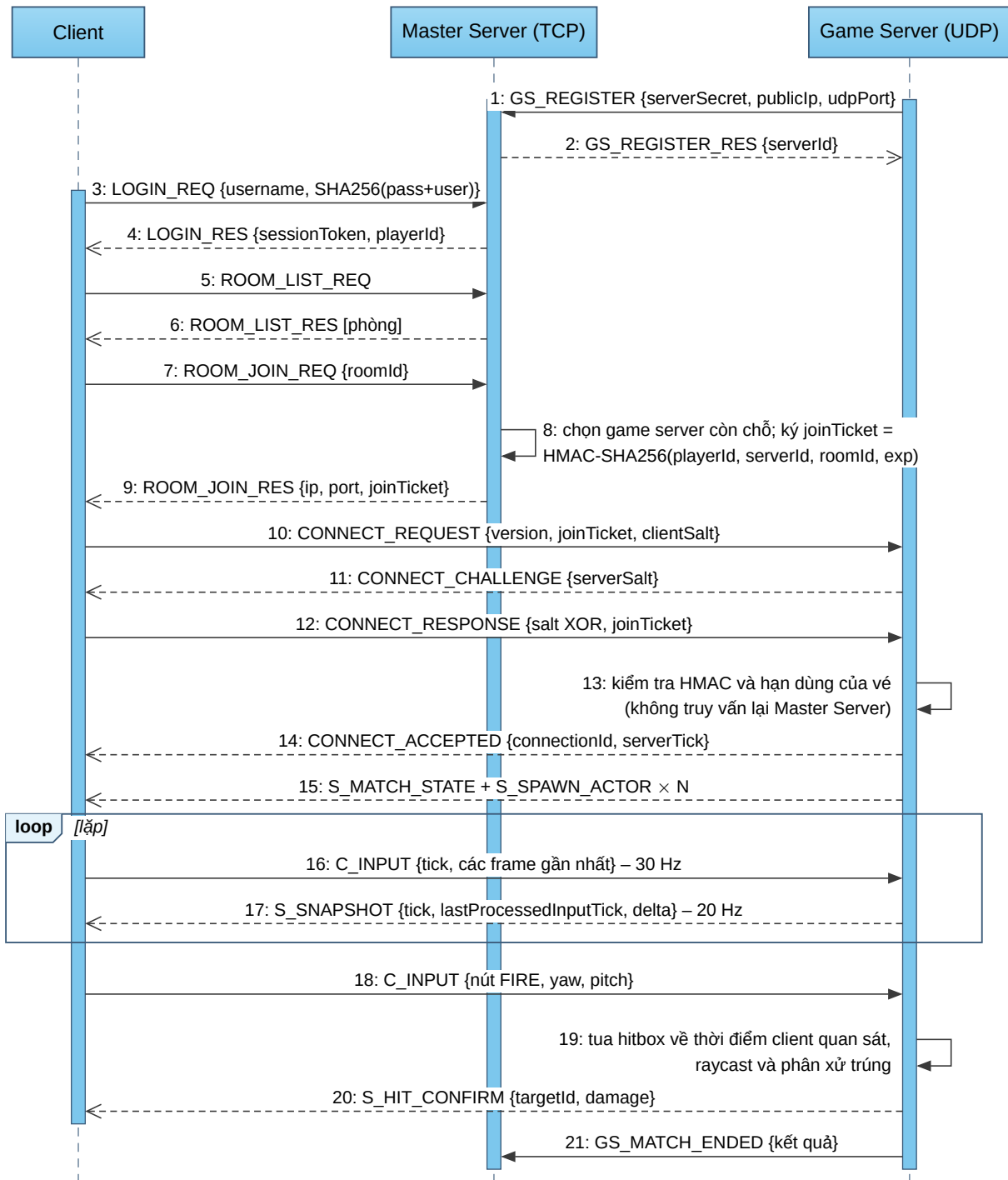
2. Mô tả thiết kế chung của hệ thống

2.1. Mô hình thẩm quyền

Hệ thống **hoàn toàn do server quyết định**: client chỉ gửi *ý định* (input), không bao giờ gửi kết quả. Nhờ vậy gian lận tốc độ, dịch chuyển tức thời, bắn nhanh hay đạn vô hạn trở nên bất khả thi về cấu trúc thay vì chỉ bị phát hiện.

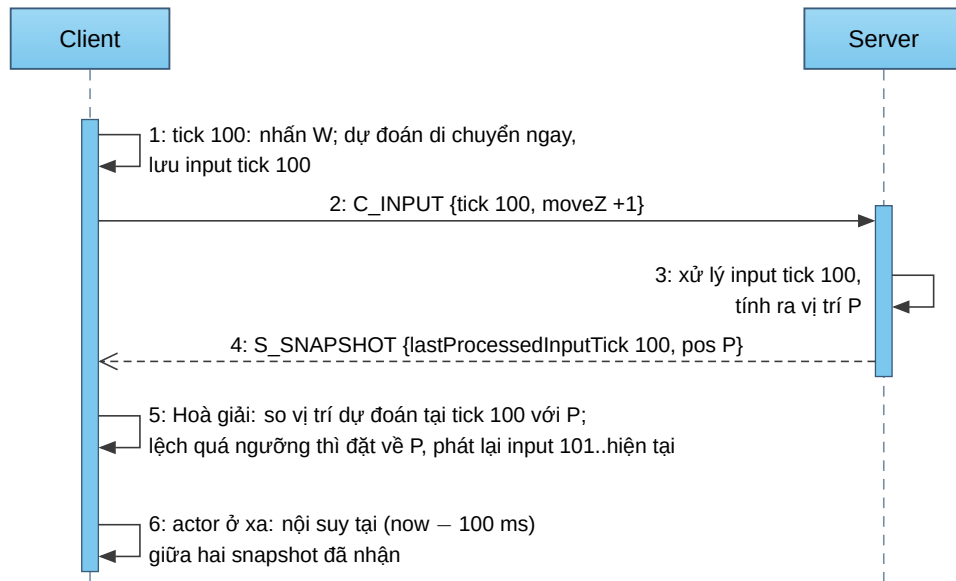
Vấn đề	Ai quyết định	Client được làm gì
Vị trí nhân vật	Server	Dự đoán ngay, sau đó hoà giải với server
Trúng đạn, sát thương	Server	Hiện hiệu ứng dự đoán, chờ S_HIT_CONFIRM
Máu, chết, hồi sinh	Server	Chỉ hiển thị
Độ tản đạn, giật súng	Server	Giật cục bộ cho cảm giác, không ảnh hưởng viên đạn thật
Bot AI, điểm chiếm, điểm số	Server	Nội suy, hiển thị
Lên xuống ghế phương tiện	Server	Gửi yêu cầu C_SEAT_REQUEST
Camera, UI, âm thanh, ragdoll	Client	Toàn quyền, không đồng bộ

2.2. Luồng nghiệp vụ đầu – cuối



Hình 2.1. Luồng đầu – cuối: đăng nhập, vào phòng, kết nối game server, chơi, kết thúc trận

2.3. Các kỹ thuật netcode cốt lõi

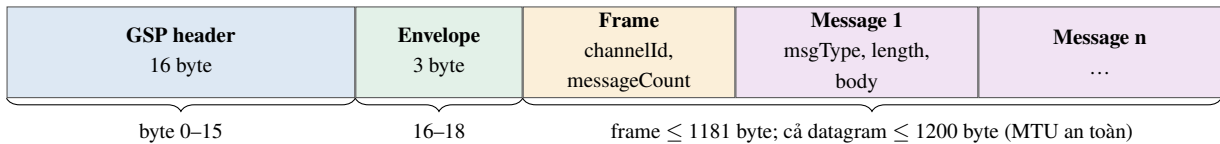


Hình 2.2. Dự đoán phía client, hoà giải với server và nội suy actor ở xa

Kỹ thuật	Mục đích	Tham số
Dự đoán phía client	Người chơi cục bộ di chuyển ngay khi nhấn phím	Cùng MovementSimulation ở client và server
Hoà giải	Sửa sai khi dự đoán lệch, phát lại input chưa xác nhận	Ngưỡng lệch 0,25 m
Nội suy thực thể	Actor ở xa chuyển động mượt dù snapshot chỉ 20 Hz	Bộ đệm 100 ms
Bù trễ	Người ping cao vẫn bắn trúng thứ họ thấy	Tua tối đa 200 ms, lịch sử hitbox 1 s

2.4. Thiết kế giao thức

2.4.1. GSP – cấu trúc một datagram PAYLOAD



Hình 2.3. Ba lớp của một datagram PAYLOAD: header GSP, envelope kênh của transport, frame message của ứng dụng

Kênh	Kiểu	Dữ liệu	Khi mất gói
0	Không tin cậy, không thứ tự	Ping/pong	Bỏ qua
1	Không tin cậy, có thứ tự (server → client)	S_SNAPSHOT, S_WEAPON_FIRE, S_VEHICLE_SNAPSHOT	Bỏ qua; gói cũ bị hủy
2	Tin cậy, có thứ tự	Sự kiện gameplay, spawn/despawn, chat, C_ACK_BASELINE	Gửi lại tối khi được ack
3	Không tin cậy, có thứ tự (client → server)	C_INPUT, C_VEHICLE_INPUT	Bỏ qua; bù bằng dư thừa

Độ tin cậy áp dụng **theo kênh**, không theo kết nối: một sự kiện bị mất trên kênh 2 không làm nghẽn snapshot trên kênh 1 — ưu điểm cốt lõi so với TCP vốn ép mọi thứ vào một luồng.

2.4.2. MSP – khung tin trên TCP

length u32, big-endian	msgType u16, little-endian	body UTF-8 JSON, tối đa 64 KB
----------------------------------	--------------------------------------	---

Hình 2.4. Khung tin MSP: tiền tố độ dài giải quyết bài toán ranh giới thông điệp trên luồng byte TCP

2.4.3. joinTicket – cầu nối TCP và UDP

playerId u32	serverId u16	roomId u16	expiresAtUnixMs u64 (60 s)	displayName 16 byte UTF-8	hmac HMAC-SHA256, 32 byte
32 byte dữ liệu được ký					32 byte chữ ký

Hình 2.5. Cấu trúc joinTicket 64 byte

Master ký vé bằng SHARED_SECRET; game server kiểm tra chữ ký bằng FixedTimeEquals và kiểm hạn dùng. Không cần gọi lại master, nên **master có thể dừng giữa trận mà trận vẫn tiếp tục**.

2.5. Tham số thời gian

Tham số	Giá trị	Ghi chú
Tick mô phỏng server	30 Hz (33,3 ms)	Theo đồng hồ thực, không theo số lần FixedUpdate
Snapshot	20 Hz (50 ms)	Rẻ hơn 30 Hz, nội suy bù độ mượt
Gửi input	30 Hz	Mỗi gói lặp lại các frame gần nhất
Bộ đệm nội suy	100 ms	Chịu được mất một snapshot
Cửa sổ bù trễ / lịch sử hitbox	200 ms / 1 s	Chặn lạm dụng ping giả
Keep-alive / timeout UDP	1 s / 10 s	
Heartbeat MSP client / game server	15 s	Phát hiện kết nối TCP nửa mở

2.6. Phân loại đối tượng đồng bộ

Loại	Ví dụ	Cách xử lý
Actor được đồng bộ	Người chơi, bot	Có actorId u16, có trong snapshot, mã hoá delta
Phương tiện	Xe, trực thăng, tàu	Luồng riêng S_VEHICLE_SNAPSHOT
Chỉ ở server	Lịch sử hitbox, dữ liệu AI	Không bao giờ gửi
Chỉ ở client	Camera, UI, particle, ragdoll, âm thanh	Không bao giờ gửi
Sự kiện một lần	Chết, nổ, phát súng, chiếm điểm	Kênh tin cậy có thứ tự
Tính	Địa hình, công trình, điểm hồi sinh	Có sẵn trong scene hai phía

2.7. Bảo mật ở mức phù hợp phạm vi

Mối đe dọa	Biện pháp	Tầng
Tăng tốc, dịch chuyển, bắn nhanh, đạn vô hạn	Server chỉ nhận input; kẹp tốc độ; tự quản lý cooldown và đạn	Replication
Mạo danh người chơi	connectionId gắn với playerId trong joinTicket đã ký	Transport
Khuếch đại giả mạo IP	Bắt tay challenge không lưu trạng thái (cookie HMAC)	Transport
Gói rác, lụt gói	Kiểm protocolId; giới hạn tốc độ theo IP; giới hạn nhóm phân mảnh	Transport
Nghe lén mật khẩu	TLS và ghim vân tay chứng chỉ; SHA-256 phía client, bcrypt phía server	Master
Tràn bộ nhớ, Slowloris	Khung tối đa 64 KB; hạn 30 s trước xác thực; 5 kết nối/IP	Master
SQL injection, dò mật khẩu	Truy vấn tham số hoá; 5 lần/phút/IP; khoá 15 phút sau 10 lần sai	Master

2.8. Các quyết định kiến trúc đã chốt

Mã	Quyết định	Lý do
AD-1	Server nắm quyền, không có listen-server	Thẩm quyền rõ ràng; tránh sửa 21 singleton
AD-2	Game server là Unity headless	Tái dùng PhysX, A*, AI; tiết kiệm 8–12 tuần
AD-3	Không theo đuổi tất định (lockstep)	Random rải rác 27 tệp; PhysX không tất định
AD-4	Ragdoll chỉ là hiệu ứng cục bộ	Tiết kiệm khoảng 1,7 MB/s
AD-5	Transport, Replication là thư viện .NET thuần	Kiểm thử bằng xUnit, tránh xung đột Unity
AD-7	Snapshot không tin cậy, sự kiện tin cậy	Đúng bản chất trạng thái và sự kiện
AD-8	Không dùng WebSocket	Yêu cầu môn học và head-of-line blocking

2.9. Quy trình phát triển và bảo đảm chất lượng

- **Đặc tả giao thức đóng băng** và `ProtocolConstants.cs` là nguồn sự thật duy nhất; `tools/SpecChecker` làm hỏng build nếu hai bên lệch nhau.
- **Tách người viết và người kiểm:** phân hệ Transport viết bit-packing, phân hệ Replication viết bộ test conformance bằng hex viết tay theo đặc tả.
- **CI** chạy trên Ubuntu và Windows: build coi cảnh báo là lỗi, chạy test, kiểm tra lệch đặc tả.

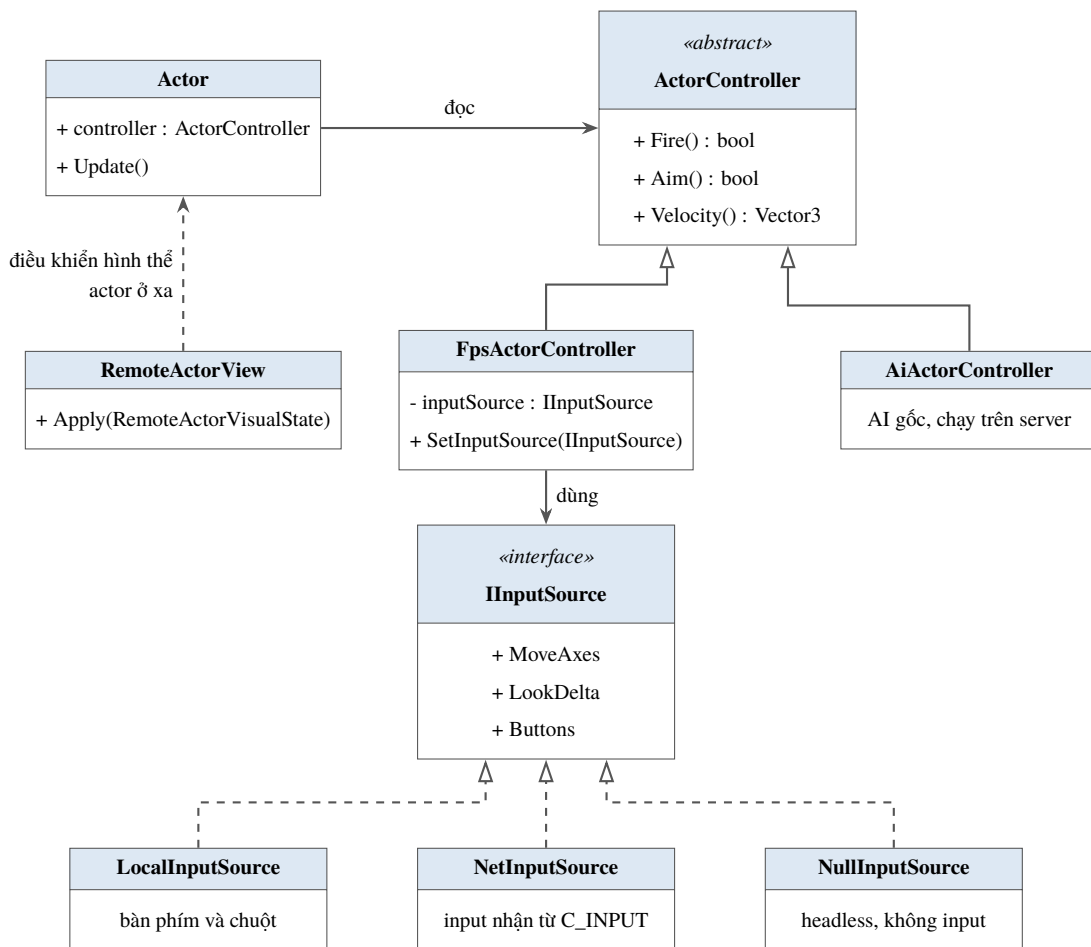
- **Definition of Done:** đã chạy và xem đầu ra, test phân hệ xanh, toàn bộ test xanh, Unity biên dịch sạch, đã merge và viết báo cáo phase.

II. NỘI DUNG CÁ NHÂN – Phân hệ client Unity.

3. Biểu đồ và giao diện nội dung cá nhân thực hiện

Phân hệ client là nơi các thư viện mạng được đưa vào trò chơi thật. Nhiệm vụ gồm: mở **điểm nối netcode** trong mã gameplay một người chơi mà không phá lối chơi gốc; làm cho **người chơi ở xa hiển thị mượt** (nội suy, hoạt ảnh, ragdoll cục bộ); làm cho **người chơi cục bộ phản hồi tức thì** (dự đoán và hoà giải); và xây dựng **giao diện** sảnh chờ, HUD, overlay chẩn đoán. Đây cũng là thành viên duy nhất mở Unity Editor.

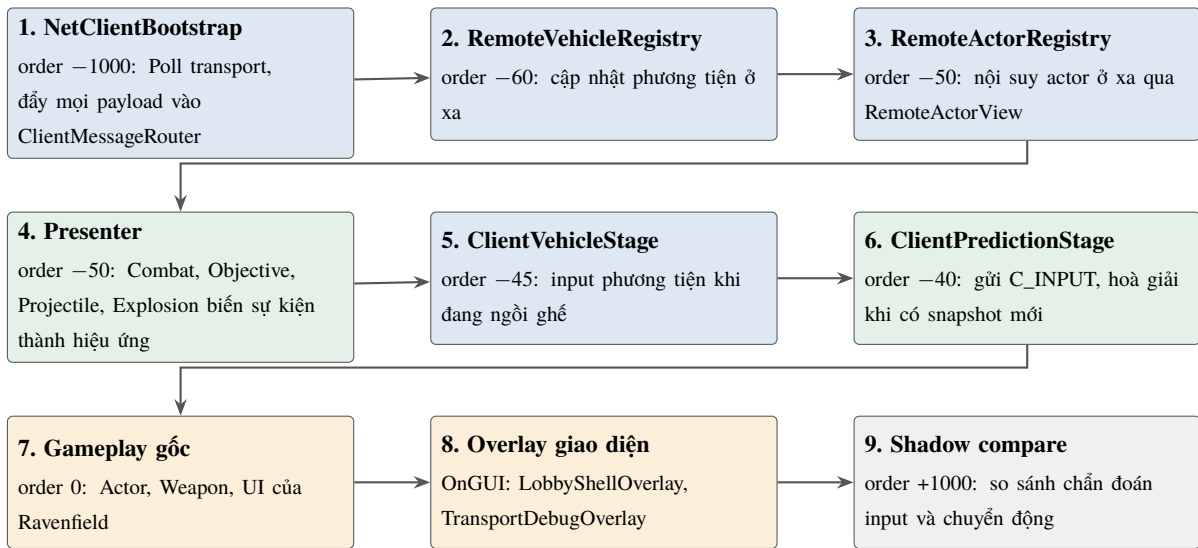
3.1. Điểm nối netcode trong mã gameplay



Hình 3.1. Biểu đồ lớp điểm nối netcode: tách nguồn input khỏi FpsActorController

`Actor.cs` không biết và không cần biết controller lấy dữ liệu từ đâu. Tách 59 lời gọi `Input.*` ra sau giao diện `IInputSource` giúp cùng một controller nhận input từ bàn phím (client), từ gói mạng (server) hoặc không nhận gì (headless).

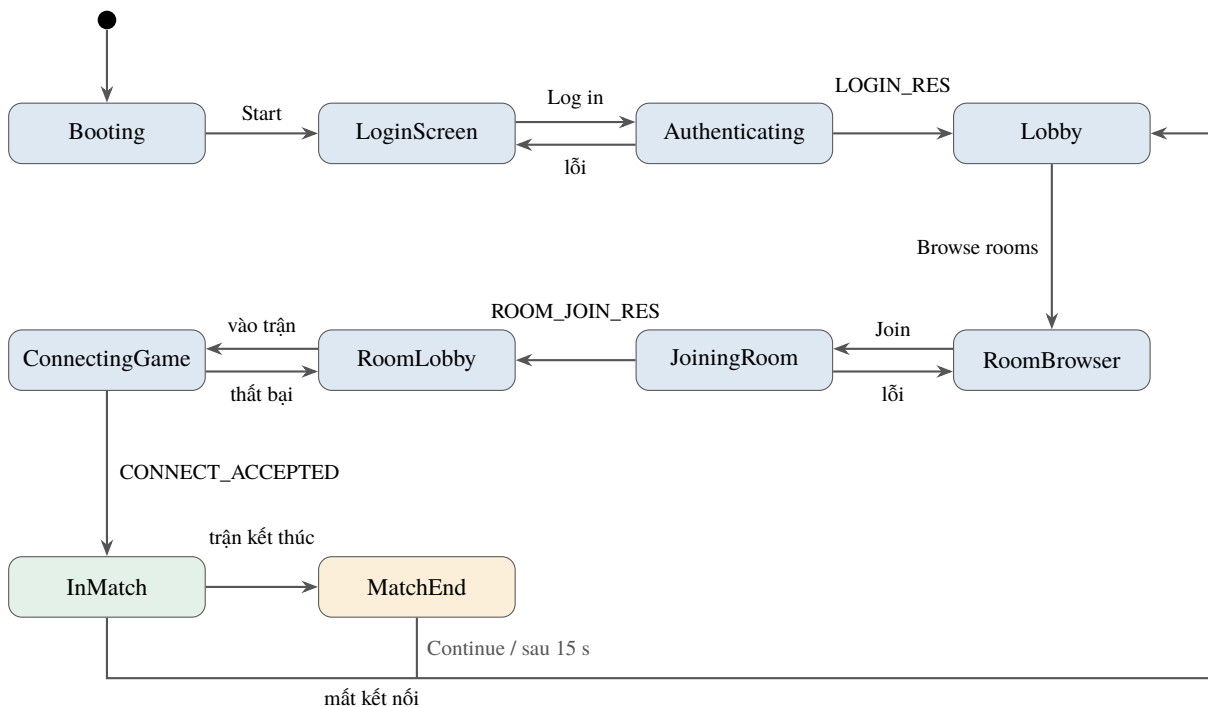
3.2. Thứ tự thực thi một khung hình client



Hình 3.2. Chuỗi thành phần client theo DefaultExecutionOrder trong một khung hình

Mọi dữ liệu đến được áp dụng **trước** khi có thành phần nào đọc nó: bootstrap chạy sớm nhất để `NetContext.Role` được gán trước mọi `Awake`, và các bước chẩn đoán chạy sau cùng.

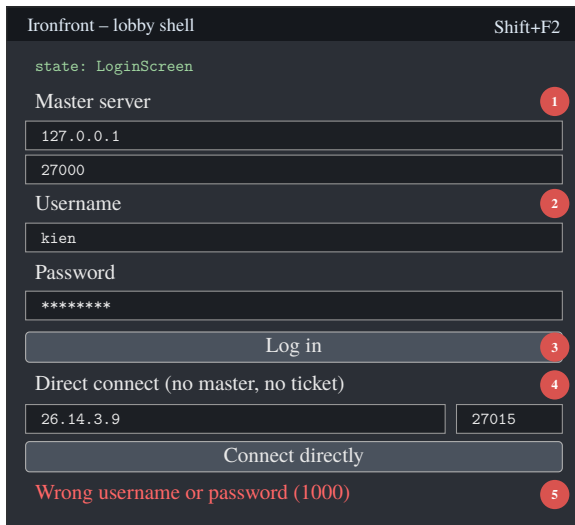
3.3. Máy trạng thái luồng trò chơi



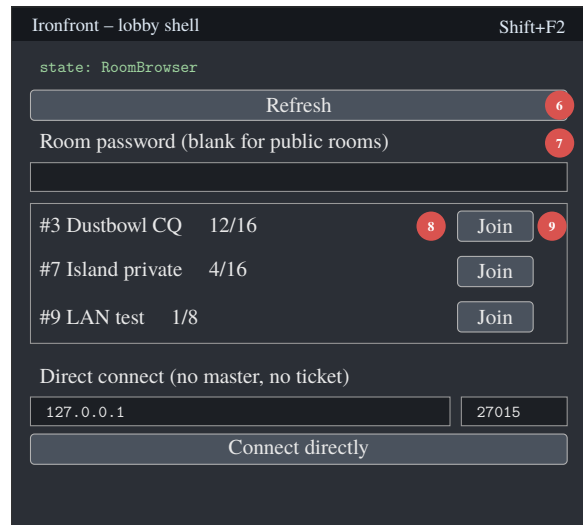
Hình 3.3. Máy trạng thái GameFlowController: 10 trạng thái; mọi chuyển trạng thái ngoài bảng đều bị từ chối

3.4. Giao diện sảnh chờ

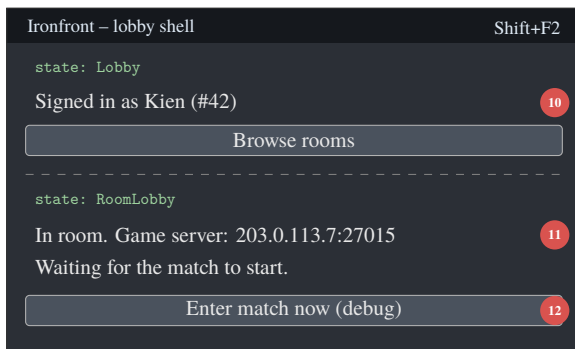
Giao diện sảnh được vẽ hoàn toàn bằng ImGui (OnGUI) trong LobbyShellOverlay, không cần Canvas hay prefab. Mỗi màn hình tương ứng một trạng thái ở Hình 3.3; dòng state: đầu khung luôn hiển thị trạng thái hiện tại. Số đỏ trên hình là mã phần tử dùng trong bảng ánh xạ ở Mục 4.



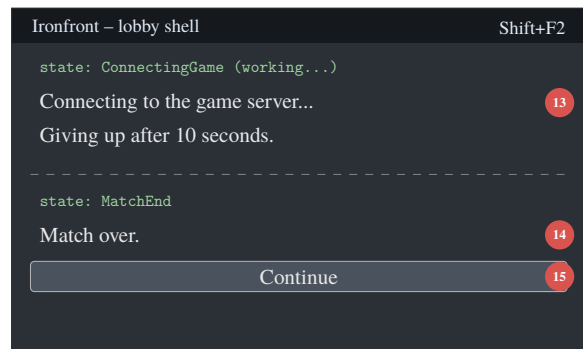
(a) LoginScreen / Authenticating



(b) Lobby → RoomBrowser / JoiningRoom



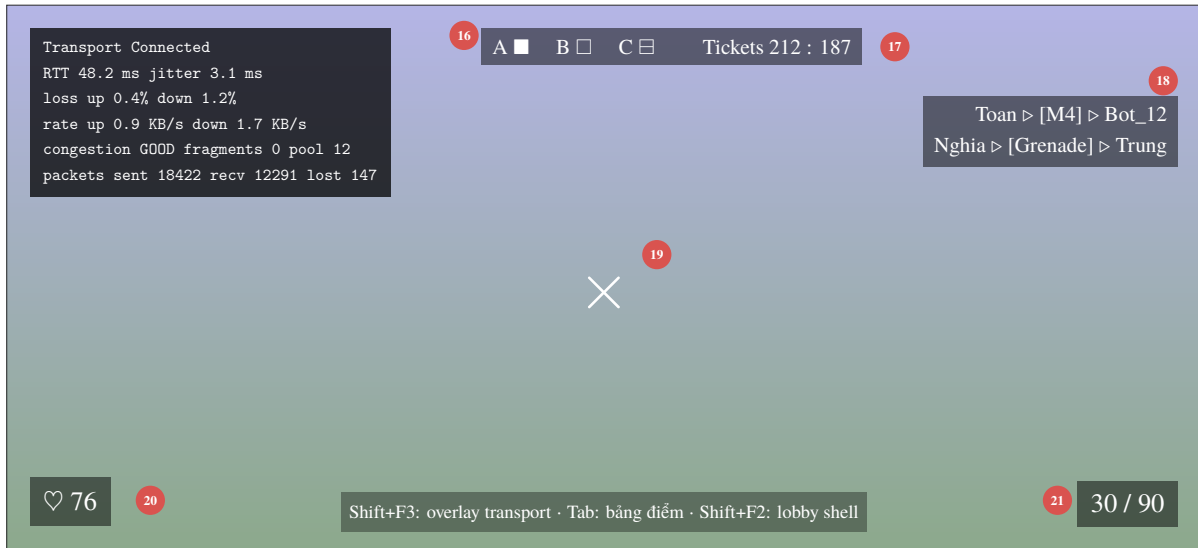
(c) Lobby và RoomLobby



(d) ConnectingGame và MatchEnd

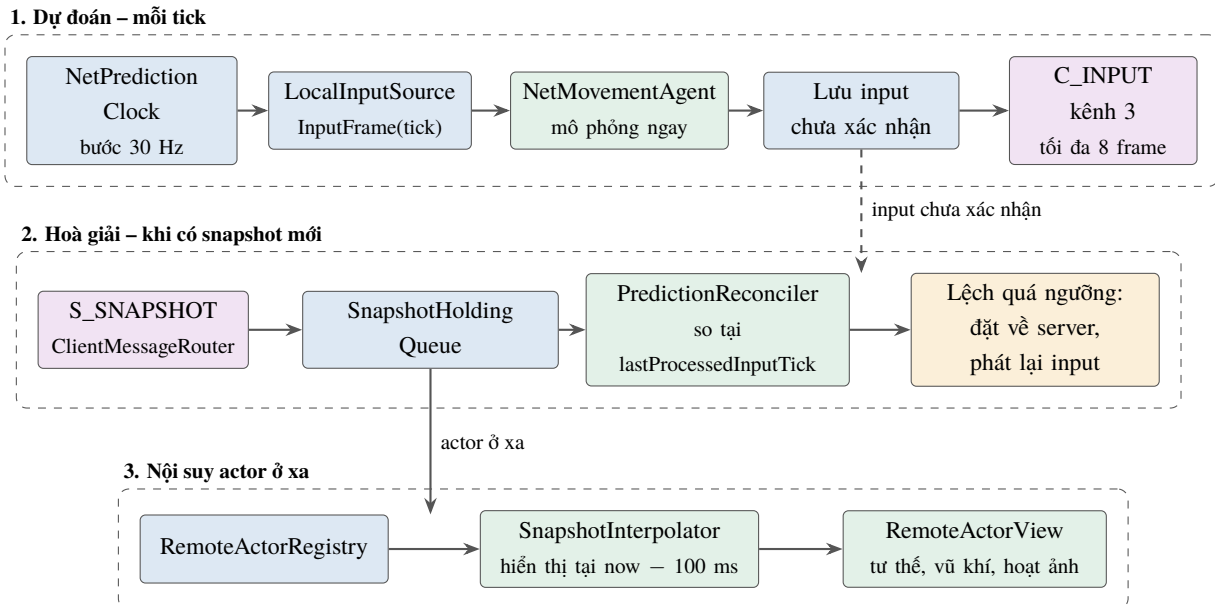
Hình 3.4. Các màn hình giao diện sảnh chờ, dựng lại theo mã LobbyShellOverlay (bổ cục ImGui)

3.5. Giao diện trong trận và overlay chẩn đoán



Hình 3.5. Bố cục màn hình trong trận: HUD được cấp dữ liệu từ server và overlay chẩn đoán transport (Shift+F3); các số là minh họa bố cục

3.6. Luồng dự đoán và hoà giải phía client



Hình 3.6. Luồng xử lý dự đoán, hoà giải và nội suy trong client

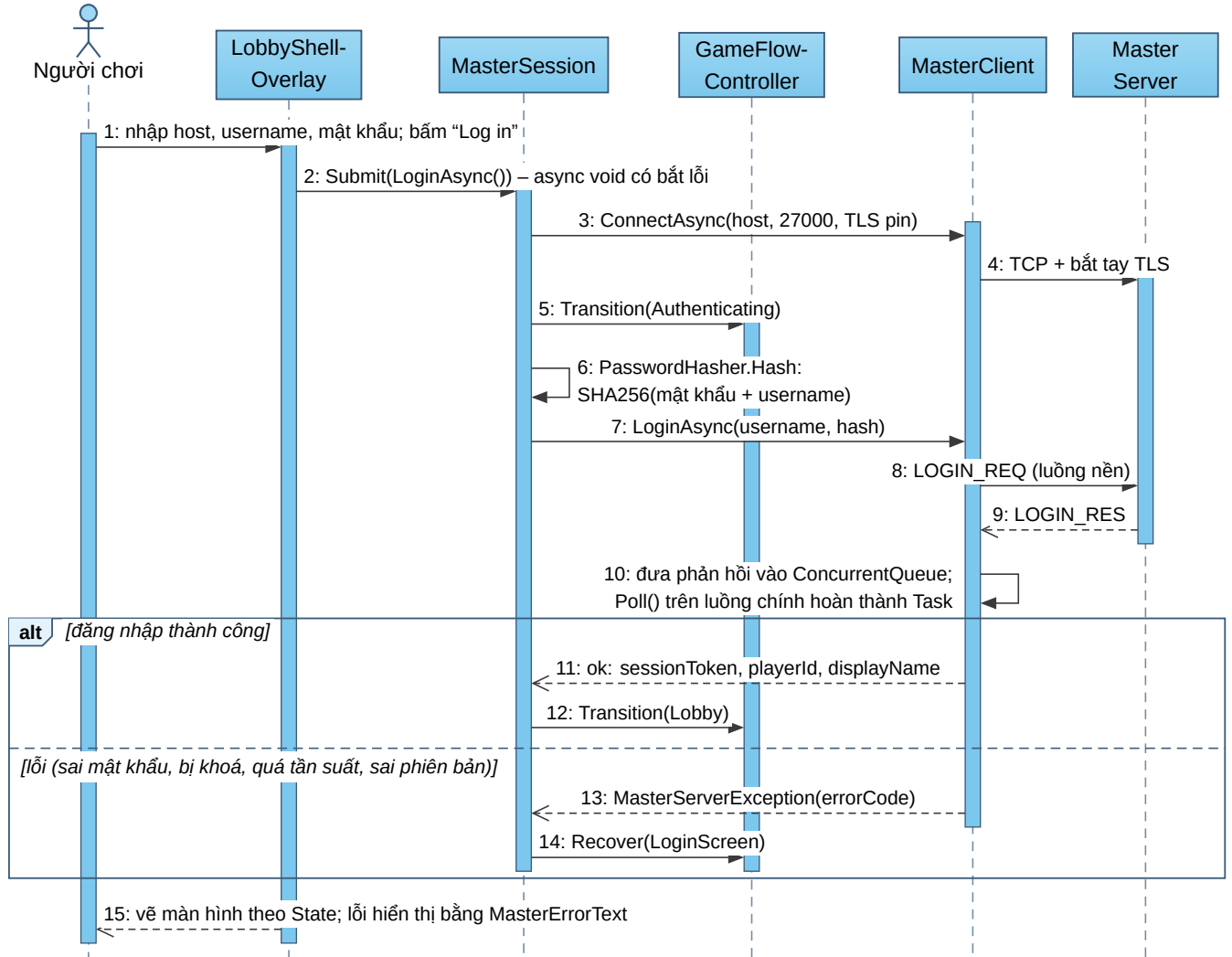
4. Luồng xử lý và giải pháp đã ứng dụng

Mục này mô tả chi tiết các luồng mà phân hệ client hiện thực: từ màn hình đăng nhập tới sảnh, vào phòng và kết nối game server bằng vé, rồi trong trận là dự đoán – hoà giải chuyển động của người chơi cục bộ, nội suy người chơi ở xa, hiển thị sự kiện chiến đấu và HUD, cuối cùng là mất kết nối và kết thúc trận. Mỗi luồng có sơ đồ tuần tự (tên lớp đúng như trong mã

nguồn), diễn giải và bảng **vấn đề – giải pháp**.

4.1. Luồng đăng nhập từ giao diện

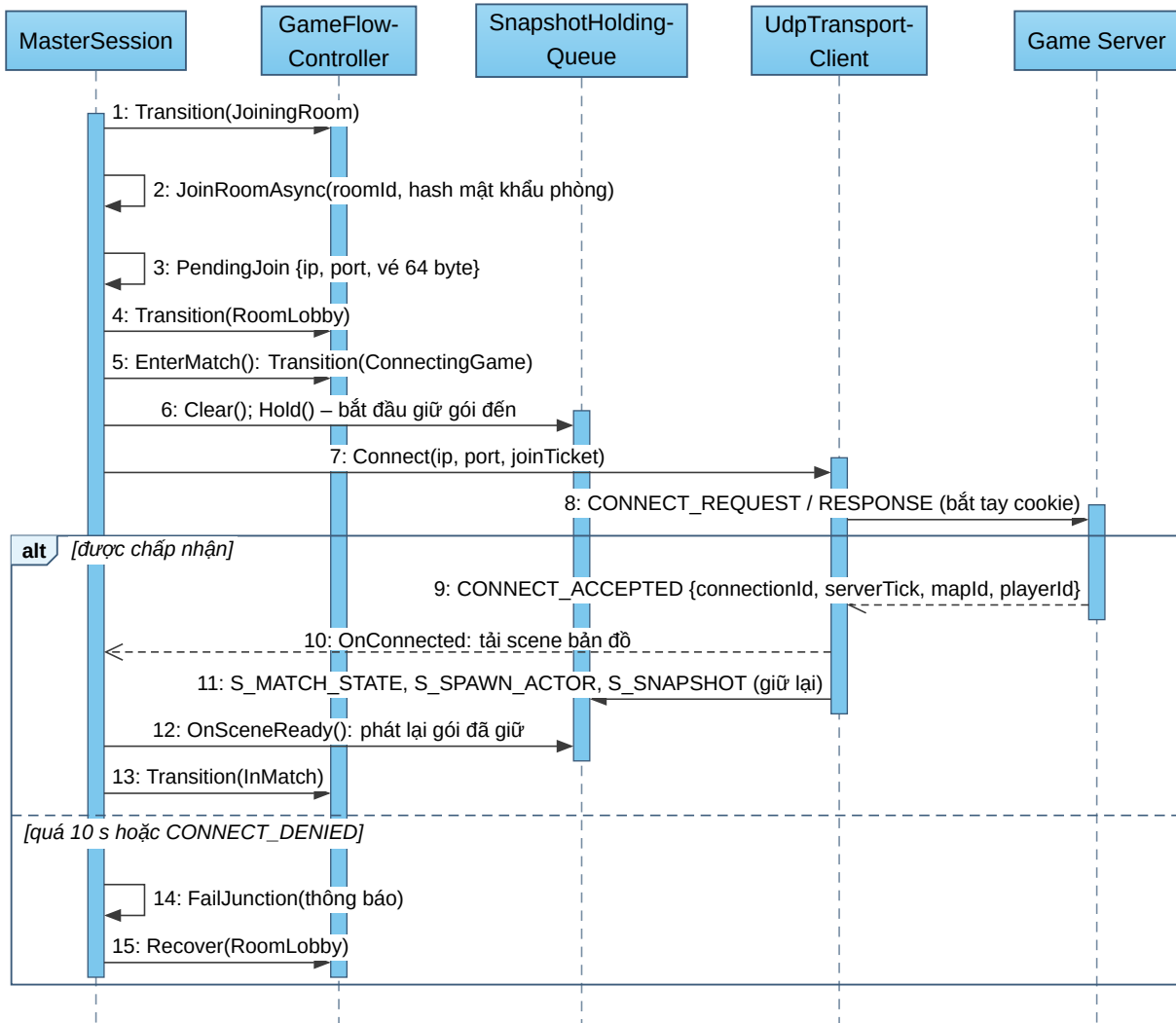
Bài toán. OnGUI chạy trên luồng chính của Unity và không thể await; nếu chờ phản hồi mạng ngay trong OnGUI thì cả trò chơi đứng hình. Ngược lại, callback mạng không được phép chạm vào đối tượng Unity từ luồng nền.



Hình 4.1. Biểu đồ tuần tự đăng nhập: giao diện không bao giờ chặn, mọi kết quả về luồng chính qua Poll()

Vấn đề	Giải pháp đã áp dụng
OnGUI không await được	Nút bấm gọi Submit(Task): một asyncvoid duy nhất có try/catch bao trọn, để ngoại lệ không thoát ra ngoài (ngoại lệ thoát khỏi asyncvoid làm sập luồng đồng bộ hoá của Unity)
Callback mạng chạy trên luồng nền	MasterClient chép khung vào ConcurrentQueue<Action>; Poll() gọi mỗi khung hình rút hàng đợi trên luồng chính
Chuyển màn hình sai thứ tự khi có nhiều thao tác chồng nhau	GameFlowController là máy trạng thái 10 trạng thái có bảng cạnh hợp lệ; Transition ngoài bảng ném lỗi, Recover chỉ đi theo cạnh phục hồi đã khai báo
Mật khẩu gốc lộ ra server hoặc log	Bấm SHA256(mật khẩu+username) ngay ở client; mật khẩu phòng bấm riêng bằng HashRoomPassword
Mã lỗi khó hiểu	MasterErrorText ánh xạ mã lỗi MSP sang câu thông báo cho người chơi

4.2. Luồng vào phòng và kết nối game server

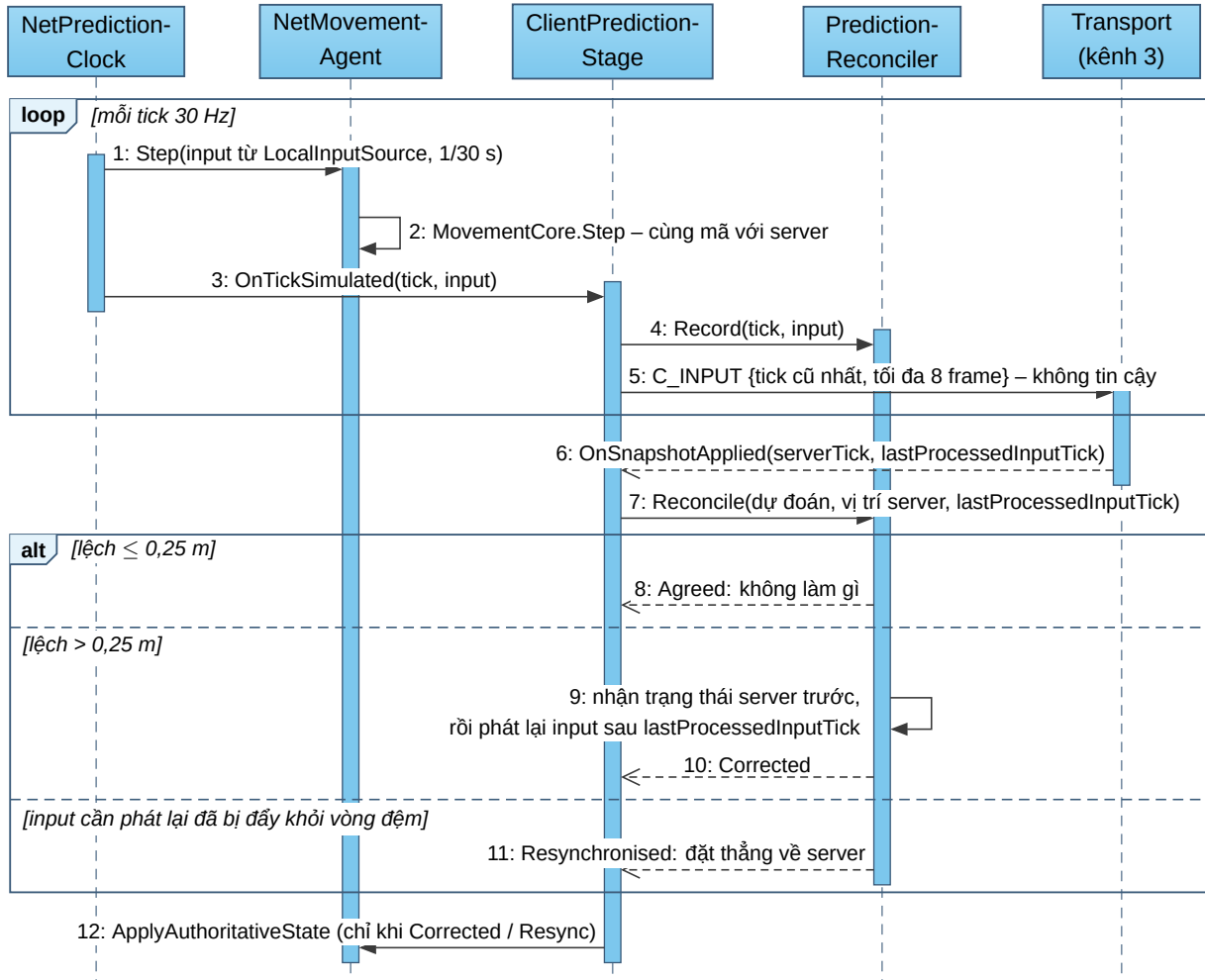


Hình 4.2. Biểu đồ tuần tự từ vào phòng tới vào trận: vé từ master, bắt tay UDP, giữ gói trong lúc tải bản đồ

Các giải pháp kỹ thuật chính.

- **PendingJoin gói địa chỉ, cổng và vé vào một giá trị.** Tránh việc gọi nhầm server này với vé của server khác (game server sẽ từ chối với lỗi chữ ký không nói rõ nguyên nhân). Vé chỉ sống 60 s nên client quay số ngay và hết hạn chờ sau 10 s — nằm gọn trong hạn của vé.
- **Giữ gói trong lúc tải bản đồ.** Server bắt đầu gửi trạng thái ngay sau CONNECT_ACCEPTED, khi scene còn đang tải. SnapshotHoldingQueue bắt đầu giữ **trước** khi quay số, và chỉ phát lại khi scene sẵn sàng; vì thế không có khoảng hở nào để một gói bị đưa vào scene chưa tồn tại, và HUD không xuất hiện trên bản đồ tải dở.
- **Lỗi quay số không làm sập khung hình.** Vé sai độ dài, cổng ngoài phạm vi hay tên máy không phân giải được đều bị bắt và biến thành thông báo, trạng thái quay về RoomLobby.
- **Kết nối trực tiếp (LAN) không cần master:** ô “Direct connect” dùng vé không ký 64 byte, dành cho thử nghiệm khi game server chạy độc lập.

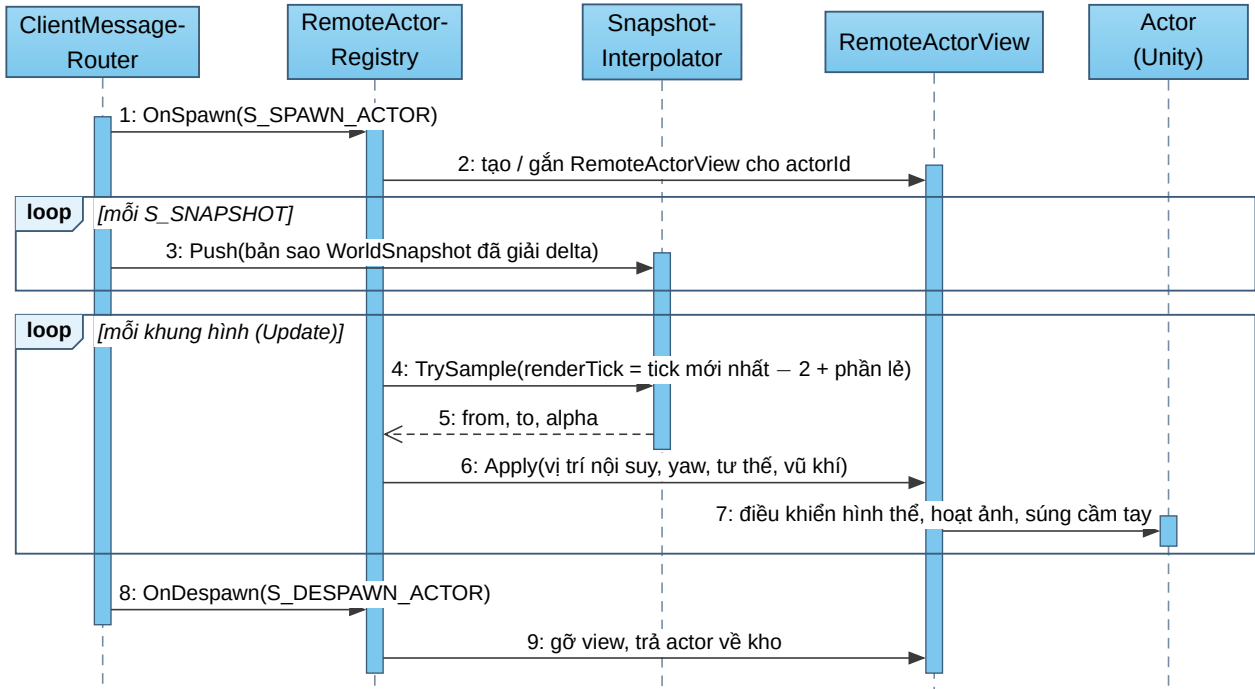
4.3. Luồng dự đoán và hoà giải chuyển động người chơi cục bộ



Hình 4.3. Biểu đồ tuần tự dự đoán phía client và hoà giải với snapshot của server

Vấn đề	Giải pháp đã áp dụng
Chờ server trả lời mới di chuyển thì trễ bằng một vòng RTT	Dự đoán: mô phỏng ngay bằng MovementCore — mã dùng chung với server nên cùng input cho cùng kết quả
Gói input bị mất	Mỗi C_INPUT mang tối đa 8 frame gần nhất (266 ms dư thừa ở 30 Hz), gửi không tin cậy trên kênh 3 — rẻ hơn trả ack cho 30 bản tin mỗi giây
Dự đoán lệch server	So tại lastProcessedInputTick; lệch quá 0,25 m thì nhận trạng thái server trước rồi mới phát lại input chưa xác nhận (phát lại lên trạng thái dự đoán sẽ nhân đôi sai số)
Ghi input sai tick làm mọi lần phát lại lệch một khung	Ghi Record trước khi gửi và dùng đúng tick mà đồng hồ đã đóng dấu
Tick 32 bit quay vòng	So sánh bằng SequenceMath.IsNewer32; ack cũ bị bỏ qua (Stale)
Ép CharacterController di chuyển thừa	Chỉ ghi lại trạng thái khi thật sự Corrected hoặc Resynchronised

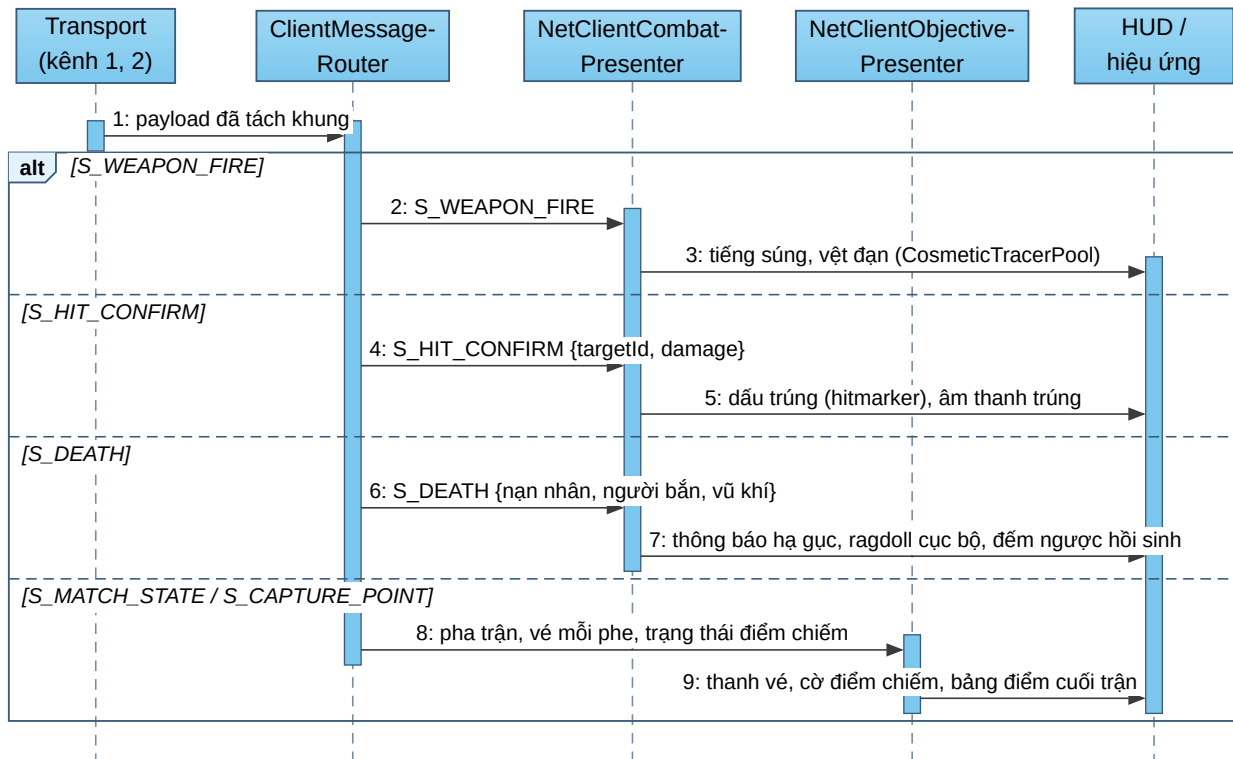
4.4. Luồng hiển thị người chơi và bot ở xa



Hình 4.4. Biểu đồ tuần tự nội suy actor ở xa giữa hai snapshot đã nhận

Giải pháp. Snapshot chỉ tới 20 Hz (xa hơn còn 10 hoặc 4 Hz) nên hiển thị thẳng sẽ giật. Client hiển thị lùi sau snapshot mới nhất 2 tick và **nội suy** giữa hai snapshot bao quanh thời điểm đó — mất một snapshot vẫn còn cặp khác để nội suy, đúng yêu cầu “chuyển động của người khác luôn mượt khi mất gói”. Vòng đệm 16 snapshot (nửa giây) được cấp phát một lần; snapshot được **chép vào** chứ không giữ tham chiếu, vì DeltaDecoder tái sử dụng cùng một đối tượng. Nếu thời điểm hiển thị cũ hơn snapshot cũ nhất thì dừng ở snapshot cũ nhất chứ không ngoại suy lùi. Ragdoll khi chết chỉ là hiệu ứng cục bộ, không đồng bộ (tiết kiệm khoảng 1,7 MB/s).

4.5. Luồng sự kiện chiến đấu, mục tiêu và HUD

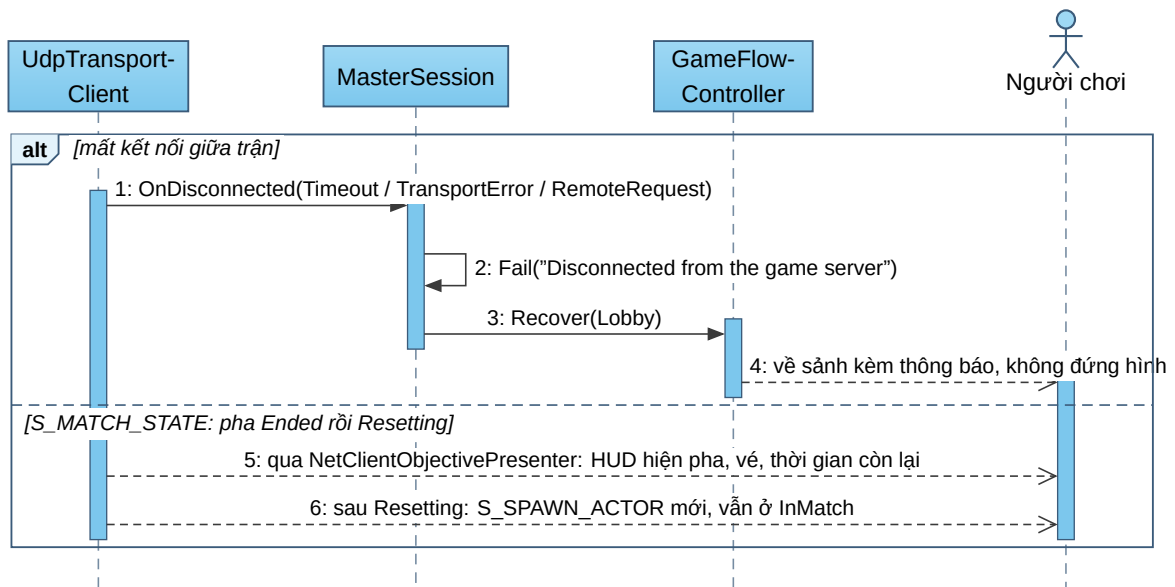


Hình 4.5. Biểu đồ tuần tự biến sự kiện server thành hiệu ứng và HUD

Máu, đạn, hạ gục và điểm số đều **do server quyết định**; client chỉ hiển thị. Giật súng cục bộ vẫn chạy để có cảm giác bắn, nhưng không ảnh hưởng viên đạn thật. Trước khi chạm vào bất kỳ đối tượng nào, mọi presenter hỏi `NetClientPresenterGuard` hai câu: “có nên chạy không” (chỉ ở vai trò client mạng) và “actor này có phải người đang ngồi trước bàn phím không” — để hiệu ứng dành cho chính người chơi (hitmarker, đếm ngược hồi sinh) không bị áp nhầm lên actor ở xa.

4.6. Luồng mất kết nối và kết thúc trận

Theo đề, hết trận server hiện bảng điểm 20 s rồi đặt lại thể giới, người chơi không phải thoát phòng; còn người chơi mất kết nối quá 10 s bị coi như rời trận. Phía client xử lý hai trường hợp như sau.



Hình 4.6. Biểu đồ tuần tự phía client khi mất kết nối và khi trận kết thúc

Khi server không nhận được gói nào trong 10 s, nó coi người chơi đã rời trận; phía client cũng tự hết hạn sau 10 s im lặng và đưa người chơi về sảnh với thông báo, thay vì để lại một thế giới đứng yên không ai cập nhật.